Prepared in accordance with IEEE 830 / ISO/IEC/IEEE 29148 SRS guidelines

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

for

Verdant

A Web-Based Study Management Platform with Visual Growth Tracking

Document Version	1.0
Document Status	Final – For Faculty Evaluation
Prepared For	GLA University
Course / Program	B.Tech – CSE Core
Prepared By	Bhavya Aggarwal (22515000038)
Project Guide	Gautam Mukherjee
Date	15 / 08 / 2026

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for **Verdant**, a web-based study management application designed to help students plan, organise, and maintain consistent study habits through a calming, distraction-free experience paired with a symbolic virtual garden. The document is intended to serve as the single source of truth for the development team, project guide, and evaluation committee regarding what the system will do, who will use it, and the constraints under which it must be built. It is written to be used for requirement analysis, UI/UX design, database and API planning, software testing, and final project documentation.

1.2 Document Conventions

Functional requirements are uniquely identified using the format FR-XX, and are grouped by module. Each requirement is written using the keyword “shall” to indicate a mandatory capability of the system, consistent with IEEE 830 / ISO/IEC/IEEE 29148 conventions. Priority levels used throughout this document are **High** (must be implemented for a working system), **Medium** (important but can be simplified if time is short) and **Low** (optional / future enhancement).

1.3 Intended Audience and Reading Suggestions

- **Students / Development Team** – to understand what to build, module by module.
- **Project Guide / Faculty Evaluator** – to assess completeness, correctness and feasibility of scope.
- **UI/UX Designer** – to design screens based on Section 4 and Section 3 workflows.
- **Database / Backend Developer** – to design schema and APIs based on Section 8 and Section 9.
- **Software Tester** – to derive test cases directly from the functional requirements in Section 3.

1.4 Project Scope

Verdant will allow students to register, plan their study schedule using a calendar and daily to-do list, run focused study sessions using a built-in Pomodoro timer, set academic goals, and track their consistency through study streaks and analytics. Every completed study session or goal contributes to the visual growth of a personal virtual garden, which acts as a calm, non-competitive reflection of the user's consistency rather than a game.

The scope is deliberately kept realistic for a college-level team project. The system will manage personal study planning, session tracking, and a symbolic growth/reward layer — it is not a classroom management system, an LMS, or a collaborative/social platform. Multi-user collaboration, real-time chat/study groups, third-party calendar sync, and mobile native apps are explicitly out of scope for the current version and are listed as future enhancements in Section 10.

1.5 References

- IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications.
- ISO/IEC/IEEE 29148:2018 – Systems and Software Engineering – Requirements Engineering.
- React.js, Node.js, Express.js and MongoDB official documentation.

- Project description document / synopsis submitted by the student team.

2. Overall Description

2.1 Product Perspective

Verdant is a new, self-contained, independent web application. It is not a modification of an existing study or productivity tool. It follows a standard three-tier web architecture (presentation, application/API, and data layer — detailed in Section 8) so that it can later be extended with features such as collaborative study groups or calendar sync as the project matures beyond the academic prototype stage.

2.2 Product Features (Summary)

- User registration, login and profile management.
- Study planner with calendar view for scheduling tasks and sessions.
- Daily to-do list for short-term task management.
- Pomodoro-style study timer with session logging.
- Goal setting with progress tracking.
- Study streak tracking (daily consistency counter).
- Progress analytics (study time, completed tasks, streak history).
- Achievement system tied to milestones.
- Daily motivational quotes/tips.
- Minimalist virtual garden that grows with consistent study activity.
- Light and dark theme support.
- Fully responsive design across devices.

2.3 User Classes and Characteristics

User Class	Description	Technical Characteristics
Student (Primary User)	General student using the platform to plan and track study activity.	Basic computer/smartphone literacy; primary end-user of all planning and garden features.
Administrator (optional, for academic demo)	Manages platform-level configuration such as motivational content and system announcements.	Trusted internal/demo user with configuration access; optional role depending on project scope.

2.4 Operating Environment

- **Client side:** Any modern web browser (Chrome, Firefox, Edge, Safari) on desktop, laptop, tablet or smartphone with an active internet connection.
- **Server side:** Node.js runtime environment hosted on a cloud platform (e.g. Render, Railway, AWS/Azure free tier) or a local server for demonstration.
- **Database:** MongoDB Atlas (cloud) or local MongoDB instance.

2.5 Design and Implementation Constraints

- Must be developed and demonstrated within a single college semester by a small student team (typically 2–4 members).
- Must primarily use free / open-source technologies to keep the project cost at zero.
- Must be responsive and usable on both desktop and mobile screen sizes.
- Garden growth logic must remain simple and rule-based (no external game engine) to stay within the academic timeline.
- The system assumes a single-user personal workspace model (no shared/collaborative garden or planner in the current version).

2.6 Assumptions and Dependencies

- Users have basic familiarity with using web forms and digital calendars.
- Reliable internet connectivity is available at both client and server ends.
- Email delivery (for reminders, if implemented) depends on a third-party SMTP service (e.g. Gmail SMTP / SendGrid free tier).
- Garden visuals (illustrations/icons) are either custom-designed or sourced from free/open-license asset libraries.

3. System Features (Functional Requirements)

This section lists the functional requirements of the system, grouped by module. Each requirement can be traced directly to a test case (see Section 7) and a screen (see Section 4).

3.1 Module: User Registration & Authentication

Description: Allows students to securely create accounts and log in.

ID	Requirement	Priority
FR-01	The system shall allow a new user to register using name, email and password.	High
FR-02	The system shall validate email format and enforce a minimum password strength on registration.	High
FR-03	The system shall authenticate registered users via email and password login.	High
FR-04	The system shall issue a secure session/token (e.g. JWT) upon successful login.	High
FR-05	The system shall allow a user to reset a forgotten password via a one-time email link/OTP.	Medium
FR-06	The system shall allow a logged-in user to log out, invalidating the active session.	Medium
FR-07	The system shall allow a user to view and update their own profile information.	Medium

3.2 Module: Study Planner & Calendar

Description: Allows users to schedule and organise study tasks over time.

ID	Requirement	Priority
FR-08	The system shall allow a user to create a study task with a title, subject, date and time.	High
FR-09	The system shall display scheduled tasks in a calendar (daily/weekly/monthly) view.	High
FR-10	The system shall allow a user to edit or delete a scheduled task.	High
FR-11	The system shall allow a user to mark a scheduled task as completed.	High
FR-12	The system shall visually distinguish completed, pending and overdue tasks on the calendar.	Medium

3.3 Module: Daily To-Do List

Description: Lightweight day-level task management separate from the long-term calendar.

ID	Requirement	Priority
FR-13	The system shall allow a user to add a to-do item for the current day.	High
FR-14	The system shall allow a user to check off, edit, or delete a to-do item.	High

ID	Requirement	Priority
FR-15	The system shall carry forward incomplete to-do items to the next day (optional reminder).	Low

3.4 Module: Pomodoro Timer

Description: Built-in focus timer for running study sessions.

ID	Requirement	Priority
FR-16	The system shall provide a configurable study timer (default 25-minute focus / 5-minute break cycle).	High
FR-17	The system shall allow a user to start, pause, resume and stop a timer session.	High
FR-18	The system shall log each completed focus session with duration, date and linked task/subject (if any).	High
FR-19	The system shall notify the user (visual/sound cue) when a focus or break interval ends.	Medium

3.5 Module: Goal Setting & Streak Tracking

Description: Lets users define study goals and tracks daily consistency.

ID	Requirement	Priority
FR-20	The system shall allow a user to create a study goal (e.g. weekly study hours, subject-based target).	High
FR-21	The system shall track progress toward each active goal automatically based on logged sessions/tasks.	High
FR-22	The system shall calculate and display the user's current daily study streak.	High
FR-23	The system shall reset the streak counter if a day passes with no qualifying study activity.	Medium
FR-24	The system shall allow a user to mark a goal as complete or archive it.	Medium

3.6 Module: Progress Analytics & Achievements

Description: Provides visual feedback on study consistency and performance over time.

ID	Requirement	Priority
FR-25	The system shall display total study time and completed tasks over selectable periods (day/week/month).	High
FR-26	The system shall display streak history in a calendar-heatmap or chart format.	Medium
FR-27	The system shall unlock achievement badges when specific milestones are reached (e.g. 7-day streak, 50 study hours).	Medium

ID	Requirement	Priority
FR-28	The system shall display a list of earned and locked achievements to the user.	Low

3.7 Module: Virtual Garden (Growth Visualization)

Description: Symbolic, non-competitive visual representation of study consistency.

ID	Requirement	Priority
FR-29	The system shall grow a sprout in the user's garden upon completion of a study session.	High
FR-30	The system shall evolve garden elements (sprout → plant → flower → tree) based on sustained streaks and milestones.	High
FR-31	The system shall unlock seasonal flower variations upon completion of weekly goals.	Medium
FR-32	The system shall display gentle animations (e.g. falling leaves, season changes) reflecting ongoing progress.	Low
FR-33	The system shall allow a user to view their garden's growth history/timeline.	Low

3.8 Module: Notifications & Daily Motivation

ID	Requirement	Priority
FR-34	The system shall display a motivational quote or study tip on the dashboard each day.	Low
FR-35	The system shall notify the user of upcoming scheduled tasks (in-app, and email if configured).	Medium
FR-36	The system shall send a reminder notification if a user's streak is at risk of breaking (e.g. no activity logged by evening).	Low

3.9 Module: Settings & Themes

ID	Requirement	Priority
FR-37	The system shall allow a user to switch between light and dark themes.	Medium
FR-38	The system shall persist the user's theme preference across sessions.	Medium
FR-39	The system shall allow a user to customise Pomodoro timer durations (focus/break length).	Low

4. External Interface Requirements

4.1 User Interfaces

- **Landing / Home Page** – brief introduction to Verdant, login/register call-to-action.
- **Dashboard** – overview of today's tasks, current streak, active timer, and garden snapshot.
- **Planner / Calendar View** – add, view, and manage scheduled study tasks.
- **Timer Screen** – Pomodoro session controls and session log.
- **Goals & Analytics View** – goal progress, study statistics, streak history, achievements.
- **Garden View** – visual display of the growing garden and growth timeline.
- **Profile & Settings** – account details, theme toggle, timer preferences.
- All screens shall follow a consistent, calm, minimal layout using the warm cream, sage green and soft beige colour palette described in the project synopsis, with rounded corners and gentle shadows.

4.2 Hardware Interfaces

No specialized hardware is required. The system runs on standard client devices (desktop/laptop/tablet/smartphone) with a working internet connection, and on any commodity server/cloud instance capable of running Node.js and MongoDB.

4.3 Software Interfaces

Component	Technology
Frontend	React.js (with React Router, Axios, and Tailwind CSS for styling)
Backend / API	Node.js with Express.js – RESTful JSON APIs
Database	MongoDB (Mongoose ODM) – NoSQL document store
Authentication	JWT (JSON Web Tokens) with bcrypt password hashing
Email Service (optional)	SMTP service (e.g. Nodemailer with Gmail/SendGrid) for reminders
Hosting (suggested)	Frontend – Vercel/Netlify; Backend – Render/Railway; Database – MongoDB Atlas

4.4 Communication Interfaces

- All client–server communication shall occur over HTTPS using RESTful JSON APIs.
- Outbound email notifications (if implemented) shall be sent via the SMTP protocol through the configured email service.
- *(Future phase)* Push notifications may be integrated for reminders on supported browsers/devices.

5. Use Case Summary

Actor	Use Case	Brief Description
Student	Register / Login	Create an account or sign in to access the platform.
Student	Schedule Study Task	Add a task to the planner/calendar with date and time.
Student	Manage Daily To-Do List	Add, complete, or remove short-term daily tasks.
Student	Run Pomodoro Session	Start a focus/break timer and log completed sessions.
Student	Set Study Goal	Define a target (e.g. weekly hours) and track progress.
Student	View Streak & Analytics	Check consistency, study time, and progress charts.
Student	View Garden	See the current state of the virtual garden and its growth history.
Student	Earn Achievement	Unlock a badge automatically upon reaching a milestone.
Student	Customize Settings	Switch theme and adjust timer preferences.

5.1 Primary Flow – Completing a Study Session

1) Student logs in → 2) Selects or creates a study task → 3) Starts the Pomodoro timer for that task → 4) Completes the focus session → 5) System logs the session, updates streak and analytics → 6) System grows or evolves an element in the virtual garden based on the updated streak/goal progress → 7) Dashboard reflects the updated streak, stats, and garden state.

6. Non-Functional Requirements

6.1 Performance Requirements

ID	Requirement	Priority
NFR-01	The system shall load any core page within 3 seconds under normal load on a standard broadband connection.	High
NFR-02	The system shall support at least 50–100 concurrent users without noticeable performance degradation (adequate for a college demo/pilot).	Medium
NFR-03	Garden state and streak updates shall be reflected within 2 seconds of completing a session or task.	Medium

6.2 Security Requirements

ID	Requirement	Priority
NFR-04	Passwords shall be stored using a one-way hashing algorithm (e.g. bcrypt); plain-text passwords shall never be stored.	High
NFR-05	All API endpoints that access personal study data shall require a valid authentication token.	High
NFR-06	All data in transit shall be encrypted using HTTPS/TLS.	High
NFR-07	User input shall be validated and sanitized on both client and server to prevent injection attacks.	High

6.3 Usability

- The interface shall be simple enough for a first-time user to schedule a task and start a timer in under 5 clicks/taps.
- The system shall be fully responsive across desktop, tablet and mobile screen sizes.
- Error and success messages shall be clear, specific, and shown close to the relevant form field/action.
- Garden growth feedback shall be gentle and non-intrusive, avoiding sudden or jarring animations.

6.4 Reliability & Availability

- The system shall target 99% uptime during the demonstration/evaluation period.
- Logged study sessions, tasks, and garden state shall not be lost due to a single server restart (persisted in the database, not in memory).

6.5 Maintainability & Portability

- The codebase shall follow a modular MVC-style structure (routes/controllers/models on the backend, reusable components on the frontend) to ease future maintenance and extension by new student contributors.

- The system shall run on any OS capable of hosting Node.js and MongoDB (Windows, Linux, macOS), and shall be accessible from any modern browser without requiring a native app install.

6.6 Scalability

Although scoped for a single-user personal workspace in the academic version, the data model and API design shall avoid hard-coded assumptions that would block future multi-user or collaborative features (e.g. shared study groups) where reasonably possible.

7. Software Testing Considerations

Each functional requirement in Section 3 shall map to at least one test case. The table below gives representative examples to guide the team's test plan; the full test case document can be maintained separately (e.g. in a spreadsheet) using this format.

Test ID	Req.	Test Scenario	Expected Result
TC-01	FR-16	Start and complete a Pomodoro focus session	Session logged with correct duration; streak updated.
TC-02	FR-29	Complete first study session as a new user	A sprout appears in the garden view.
TC-03	FR-22	Log activity on consecutive days	Streak counter increments correctly each day.
TC-04	FR-23	Skip a day with no logged activity	Streak resets to zero on the following login.
TC-05	FR-02	Register with a weak password	Registration blocked with validation message.

Recommended testing types: **Unit testing** (backend controllers/validators using a framework such as Jest/Mocha), **Integration testing** (API endpoints with Postman/Jest+Supertest), and **Manual UI/UX testing** across at least two browsers and one mobile viewport before final submission.

8. Proposed System Architecture

Given the team's familiarity and the project's realistic academic scope, a **MERN stack** (MongoDB, Express.js, React.js, Node.js) three-tier architecture is recommended. This keeps the project fully JavaScript-based end-to-end, is well documented, free to host on student-friendly free tiers, and maps cleanly onto the module breakdown in Section 3.

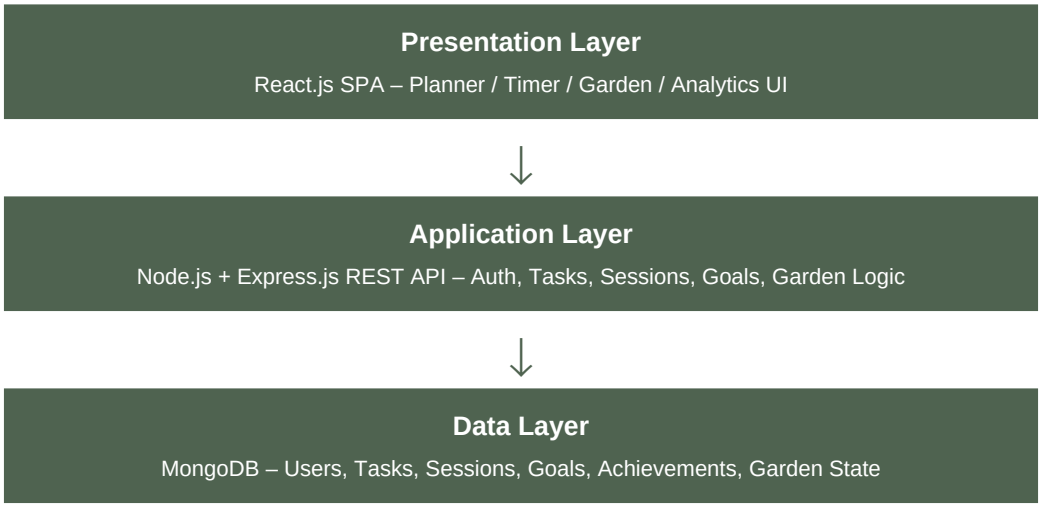

Figure 8.1: Three-tier architecture of Verdant.

8.1 Suggested Technology Stack

Layer	Technology
Frontend	React.js, React Router, Axios, Tailwind CSS
Backend	Node.js, Express.js, JWT, bcrypt, Nodemailer (optional)
Database	MongoDB with Mongoose
Version Control	Git & GitHub
Testing	Postman (API testing), Jest (unit testing)
Deployment (suggested)	Vercel/Netlify (frontend), Render/Railway (backend), MongoDB Atlas (database)

9. Data Model Overview (for Database Planning)

The following core entities and relationships are proposed as a starting point for schema design. This is intentionally kept minimal and realistic for a student project; fields can be extended during implementation.

9.1 Core Entities

Entity	Key Attributes
User	_id, name, email, passwordHash, theme, createdAt
Task	_id, userId, title, subject, date, time, status (Pending/Completed/Overdue), createdAt
StudySession	_id, userId, taskId (optional), duration, startedAt, completedAt
Goal	_id, userId, title, targetType (hours/tasks), targetValue, currentProgress, status (Active/Completed/Archived)
Streak	_id, userId, currentStreak, longestStreak, lastActiveDate
Achievement	_id, userId, title, description, unlockedAt
GardenState	_id, userId, growthStage, plantType, unlockedElements[ ], lastUpdated
Notification	_id, userId, message, type, isRead, createdAt

9.2 Key Relationships

- A **User** has one **Streak** record and one **GardenState** record (one-to-one).
- A **User** has many **Tasks**, many **StudySessions**, many **Goals**, and many **Achievements** (one-to-many).
- A **Task** may have many linked **StudySessions** (one-to-many, optional link).
- Completed **StudySessions** and **Goals** drive updates to **Streak** and **GardenState** (derived/computed relationship, not a direct foreign key).

10. Future Enhancements (Out of Current Scope)

- Collaborative/shared study groups with a shared garden view.
- Third-party calendar sync (Google Calendar/Outlook).
- Native mobile applications (iOS/Android).
- Push notifications for reminders and streak alerts.
- AI-based study schedule suggestions based on past consistency.
- Exportable progress reports (PDF).

11. Glossary

Term	Definition
SRS	Software Requirements Specification
API	Application Programming Interface
REST	Representational State Transfer – a web API architecture style
JWT	JSON Web Token – used for stateless authentication
MERN	MongoDB, Express.js, React.js, Node.js – a full JavaScript web stack
CRUD	Create, Read, Update, Delete – basic data operations
OTP	One-Time Password
UI/UX	User Interface / User Experience
Streak	The count of consecutive days a user has logged qualifying study activity
Garden State	The stored representation of a user's current growth stage and unlocked visual elements

End of Document – Verdant SRS v1.0